

MCP

Model Context Protocol

Complete Study Guide

Universal Standard for Connecting LLMs to Tools, Data, and the World

Data Science & AI Engineering Prep

1. What is MCP and Why Does It Exist?

MCP (Model Context Protocol) is an open standard created by Anthropic in 2024 for connecting LLMs to external tools, data sources, and services. Before MCP, every team that wanted their AI to read files, query databases, or call APIs built their own custom connectors — different formats, different protocols, no standards, no reuse.

MCP solves this by providing a universal protocol that any LLM application can use to connect to any tool or data source — without custom integration code.

1.1 The USB Analogy

Think of MCP like USB for AI. Before USB, every device had its own connector — printers, keyboards, mice, all different. USB standardized it: one protocol, plug anything in, it just works. MCP does the same for AI tools. Build an MCP server once, connect it to any LLM application that supports MCP.

Before MCP	After MCP
Every team builds custom connectors	One MCP server works with any MCP client
Different formats, no standards	Universal JSON-RPC protocol
Can't reuse across projects	Build once, use anywhere
Fragile, hard to maintain	Standardized, easy to maintain
Like every device having its own port	Like USB — one standard for everything

1.2 What MCP Enables

- LLMs that can read and write files on your computer
- LLMs that can query databases (SQLite, PostgreSQL, MongoDB)
- LLMs that can search the web in real time
- LLMs that can interact with GitHub, Slack, Google Drive, and other services
- LLMs that can run code, take screenshots, and control browsers
- All of this through a single, standardized protocol — no custom glue code

2. MCP Architecture — 3 Core Components

Every MCP setup has three parts: a Client, a Server, and the Protocol connecting them.

Component	Role	Examples
MCP Client	The LLM app that wants to use tools	Claude Desktop, your Python app, VS Code
MCP Server	Exposes tools/resources via MCP protocol	File server, DB server, web search server
MCP Protocol	Standard language both sides speak	JSON-RPC 2.0 over stdio or HTTP/SSE

2.1 Communication Flow

MCP Client (LLM App)	MCP Protocol	MCP Server (Tool)
1. List available tools	--> list_tools()	--> Returns tool definitions
2. LLM decides to call	--> call_tool()	--> Executes the function
3. Server returns result	<-- TextContent	<-- Result as text/JSON
4. LLM reads result and generates final answer		

2.2 What MCP Servers Expose — 3 Types

Type	Description + Example
Tools	Functions the LLM can call with parameters. Example: search_web(query), read_file(path), query_db(sql)
Resources	Data the LLM can read by URI. Example: file://docs/report.pdf, db://customers/table
Prompts	Reusable prompt templates the LLM can invoke. Example: summarize_document(text), analyze_data(csv)

3. Building an MCP Server in Python

3.1 Installation

```
pip install mcp

# Also useful for connecting MCP to LangChain agents:
pip install langchain-mcp-adapters langgraph langchain-groq
```

3.2 Minimal MCP Server — Calculator

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
import asyncio

# Step 1: Create the server with a name
app = Server("calculator-server")

# Step 2: Register available tools
# The LLM reads this list to know what tools are available
@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="add_numbers",
            description="Adds two numbers together", # LLM reads this
            inputSchema={
                "type": "object",
                "properties": {
                    "a": {"type": "number", "description": "First number"},
                    "b": {"type": "number", "description": "Second number"}
                },
                "required": ["a", "b"] # Both params are required
            }
        )
    ]

# Step 3: Handle tool execution — called when LLM invokes a tool
@app.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "add_numbers":
        result = arguments["a"] + arguments["b"]
        return [TextContent(type="text", text=str(result))]

# Step 4: Run the server via stdio transport
async def main():
    async with stdio_server() as (read_stream, write_stream):
        await app.run(read_stream, write_stream,
                     app.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())
```

3.3 Practical MCP Server — File Reader + SQLite Query

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
import asyncio, json, os, sqlite3

app = Server("data-server")

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="read_file",
            description="Read contents of a text file from disk",
            inputSchema={
                "type": "object",
                "properties": {"filepath": {"type": "string", "description": "Path
to the file"}},
                "required": ["filepath"]
            },
        ),
        Tool(
            name="query_sqlite",
            description="Run a SELECT query on a SQLite database",
            inputSchema={
                "type": "object",
                "properties": {
                    "db_path": {"type": "string", "description": "Path to the
SQLite .db file"},
                    "query": {"type": "string", "description": "SQL SELECT query
to run"}
                },
                "required": ["db_path", "query"]
            },
        )
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:

    if name == "read_file":
        filepath = arguments["filepath"]
        if not os.path.exists(filepath):
            return [TextContent(type="text", text=f"File not found: {filepath}")]
        with open(filepath, "r") as f:
            content = f.read()
        return [TextContent(type="text", text=content)]

    elif name == "query_sqlite":
        conn = sqlite3.connect(arguments["db_path"])
        cursor = conn.cursor()
        cursor.execute(arguments["query"])
        rows = cursor.fetchall()
```

```
        conn.close()
        return [TextContent(type="text", text=json.dumps(rows, indent=2))]

async def main():
    async with stdio_server() as (read_stream, write_stream):
        await app.run(read_stream, write_stream,
                      app.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())
```

4. Connecting MCP Servers to Claude Desktop

Claude Desktop can connect to any MCP server by adding it to its configuration file. Once added, Claude automatically discovers the tools and uses them when relevant.

4.1 Configuration File Location

```
# macOS
~/Library/Application Support/Claude/claude_desktop_config.json

# Windows
%APPDATA%/Claude/claude_desktop_config.json
```

4.2 Registering Your Own MCP Server

```
{
  "mcpServers": {
    "calculator": {
      "command": "python",
      "args": ["C:/path/to/calculator_server.py"]
    },
    "data-server": {
      "command": "python",
      "args": ["C:/path/to/data_server.py"]
    }
  }
}

# After saving: restart Claude Desktop
# Claude will now show your tools as available in every conversation
```

4.3 Using Pre-Built MCP Servers (npm packages)

```
# Many MCP servers are published as npm packages – no coding needed
{
  "mcpServers": {

    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
        "C:/Users/Gurleen/Documents"]
    },

    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "your_key_here" }
    },

    "github": {
```

```
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-github"],
    "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "your_token_here" }
  }
}
}
```


5. Using MCP Tools in Python Apps — LangChain Integration

You don't need Claude Desktop to use MCP tools. You can connect to MCP servers directly from your Python code using LangChain's MCP adapters and build agents that automatically use the tools.

5.1 Full Agent with MCP Tools

```
from langchain_mcp_adapters.tools import load_mcp_tools
from langchain_groq import ChatGroq
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
import asyncio

async def main():
    # Step 1: Define which MCP server to connect to
    server_params = StdioServerParameters(
        command="python",
        args=["calculator_server.py"]
    )

    # Step 2: Connect and create a session
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Step 3: Load all tools the MCP server exposes
            tools = await load_mcp_tools(session)
            print(f'Loaded tools: {[t.name for t in tools]}')

            # Step 4: Create a ReAct agent — reasons about when to use tools
            llm = ChatGroq(model="llama3-8b-8192")
            agent = create_react_agent(llm, tools)

            # Step 5: Ask the agent a question
            response = await agent.ainvoke({
                "messages": [{"role": "user",
                               "content": "What is 347 + 891?"}]
            })
            print(response["messages"][-1].content)

asyncio.run(main())

# What happens internally:
# 1. Agent reads tool list: ['add_numbers']
# 2. Decides: I need add_numbers(a=347, b=891)
# 3. MCP server executes: 347 + 891 = 1238
# 4. Agent answers: '347 + 891 = 1238'
```

5.2 The Full MCP Request-Response Flow

```
User asks: 'What is 347 + 891?'
|
v
LLM reads available tools from MCP server (list_tools)
|
v
LLM decides: I need add_numbers(a=347, b=891)
|
v
MCP Client sends tool call request to MCP Server
|
v
MCP Server executes: 347 + 891 = 1238
|
v
MCP Server returns TextContent: '1238'
|
v
LLM generates final answer: '347 + 891 = 1238'
```

6. Transport Methods — stdio vs HTTP/SSE

MCP supports two transport methods for communication between client and server. The choice depends on whether the server runs locally or remotely.

stdio (Standard I/O)	HTTP / SSE (Server-Sent Events)
Client launches server as subprocess	Server runs as a standalone HTTP service
Communicates via stdin/stdout pipes	Communicates over HTTP network requests
Only accessible on same machine	Accessible from any machine on the network
Simple setup — just a Python script	Requires deploying a web server
Best for local tools and personal use	Best for team tools and cloud deployment
Used in Claude Desktop config	Used for shared / remote MCP servers

6.1 stdio Server (Local — What We Built)

```
from mcp.server.stdio import stdio_server

async def main():
    # Client spawns this process and communicates via stdin/stdout
    async with stdio_server() as (read_stream, write_stream):
        await app.run(read_stream, write_stream,
                      app.create_initialization_options())
```

6.2 HTTP/SSE Server (Remote — Cloud Deployment)

```
from mcp.server.sse import SseServerTransport
from starlette.applications import Starlette
from starlette.routing import Route

# Create SSE transport — clients connect via HTTP
transport = SseServerTransport("/messages")

# Wrap in a Starlette/FastAPI app and deploy to cloud
# Team members connect to: http://your-server.com/messages
```

7. Pre-Built MCP Servers — Ready to Use

The MCP ecosystem has hundreds of pre-built servers covering almost every tool and service. You simply add them to your Claude Desktop config or connect from Python — no server code needed.

MCP Server	What Your LLM Gets	Install
filesystem	Read/write files and folders on your computer	@modelcontextprotocol/server-filesystem
sqlite	Query SQLite databases with natural language	@modelcontextprotocol/server-sqlite
github	Search repos, read code, create issues and PRs	@modelcontextprotocol/server-github
brave-search	Real-time web search results	@modelcontextprotocol/server-brave-search
google-maps	Location search, directions, geocoding	@modelcontextprotocol/server-google-maps
postgres	Query PostgreSQL databases	@modelcontextprotocol/server-postgres
slack	Read channels, send messages, search Slack	@modelcontextprotocol/server-slack
puppeteer	Control a browser, click, screenshot, scrape	@modelcontextprotocol/server-puppeteer
memory	Persistent memory across conversations	@modelcontextprotocol/server-memory
sequential-thinking	Step-by-step reasoning for complex problems	@modelcontextprotocol/server-sequential-thinking

8. inputSchema — Why It Matters

Every MCP tool must define an `inputSchema` — a JSON Schema object describing the parameters the tool accepts. This serves two purposes: it tells the LLM what parameters to pass when calling the tool, and it validates that the correct data types are provided.

8.1 inputSchema Structure

```
Tool(
  name="search_documents",
  description="Search documents by keyword and filter by year", # LLM reads this
  inputSchema={
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "Search keyword or phrase" # LLM reads this too
      },
      "year": {
        "type": "integer",
        "description": "Filter results to this year (e.g. 2023)"
      },
      "max_results": {
        "type": "integer",
        "description": "Maximum number of results to return",
        "default": 10
      }
    },
    "required": ["query"] # Only query is required, year and max_results are
optional
  }
)

# The LLM reads name, description, and inputSchema to understand:
# - WHEN to call this tool (description)
# - WHAT parameters to pass (inputSchema properties)
# - WHICH parameters are mandatory (required array)
```

8.2 Writing Good Tool Descriptions

Bad Description	Good Description
"Does stuff with files"	"Read the full text content of any file given its absolute path"
"Query the database"	"Execute a SQL SELECT query on the SQLite database and return results as JSON"
"Search"	"Search the web for real-time information and return top 5 relevant results with URLs"

The LLM uses the description to decide WHEN to call a tool. Vague descriptions lead to the LLM not knowing when to use it. Specific, action-oriented descriptions help the LLM call the right tool at the right time.

9. Q&A Cards — Test Your Understanding

Q1

What problem does MCP solve and why was it created?

Before MCP, every team building AI applications that needed to connect to tools (files, databases, APIs) built their own custom connectors — different formats, no standards, couldn't be reused across projects. MCP (Model Context Protocol) provides a universal standard so any LLM can connect to any tool through one consistent protocol. Build an MCP server once, use it with any MCP-compatible client.

Q2

What are the 3 components of an MCP setup and what does each do?

MCP Client: the LLM application that wants to use tools (Claude Desktop, your Python app). MCP Server: the program that exposes tools/resources via the MCP protocol (a file reader, database querier, web searcher). MCP Protocol: the standard JSON-RPC 2.0 communication language that both sides speak over stdio or HTTP/SSE.

Q3

What are the 3 things an MCP Server can expose to an LLM?

Tools: callable functions with parameters, like `search_web(query)` or `read_file(path)` — the LLM actively invokes these. Resources: data the LLM can read by URI, like `file://report.pdf` or `db://customers/table`. Prompts: reusable prompt templates the LLM can invoke, like `summarize_document(text)`.

Q4

In an MCP server, what do `list_tools()` and `call_tool()` each do?

`list_tools()` returns the catalog of tools this server exposes — their names, descriptions, and `inputSchemas`. The MCP client calls this on startup to discover available tools. `call_tool()` is invoked when the LLM actually wants to use a tool — it receives the tool name and arguments, executes the function, and returns the result as `TextContent`.

Q5

What is `inputSchema` and why does the LLM need it?

`inputSchema` is a JSON Schema definition attached to each tool that describes the parameters the tool accepts — their names, types, descriptions, and which are required. The LLM reads `inputSchema` to know what parameters to pass when calling the tool. Without it, the LLM has no way to know the tool needs an argument called 'query' of type string. Good descriptions inside `inputSchema` also help the LLM understand each parameter's purpose.

Q6

What is the difference between `stdio` and `HTTP/SSE` transport?

`stdio`: the MCP client spawns the server as a subprocess and communicates via standard input/output pipes. Simple, local-only, used for personal tools and Claude Desktop integrations. `HTTP/SSE`: the server runs as a deployed HTTP service; clients connect over the network. Used when multiple people need to share the same MCP server, or when the server runs in the cloud.

Q7

How does an LLM agent decide when to call an MCP tool?

The agent reads the tool's name and description to understand its purpose, and the `inputSchema` to know required parameters. When the user's query matches what a tool can do, the LLM generates a tool call

with the appropriate arguments. For example, if the user asks '347 + 891?' and there's an `add_numbers` tool, the LLM calls `add_numbers(a=347, b=891)`, receives 1238, and answers the user.

Q8
You want to give Claude Desktop the ability to search your company's internal knowledge base. What would you build?

Build an MCP Server in Python with a `search_knowledge_base` tool. The tool takes a query string, searches your internal docs (vector search or keyword), and returns results as `TextContent`. Register the server in `claude_desktop_config.json` with the command and path to your Python script. Restart Claude Desktop — it will now use your tool automatically when asked questions about internal knowledge.

Q9
What is a ReAct agent and how does it work with MCP tools?

A ReAct (Reason + Act) agent loops between thinking and acting. It reads the available MCP tools, reasons about which tool to use for the current step, calls that tool, observes the result, then reasons about the next step. This loop continues until the agent has enough information to answer the user. `create_react_agent()` from `LangGraph` builds this loop automatically when given an LLM and a list of tools.

Q10
Name 5 pre-built MCP servers and what capability each adds to an LLM.

`filesystem`: read and write files on the local computer. `brave-search`: real-time web search results. `github`: search repositories, read code, create issues. `sqlite`: query SQLite databases with natural language. `puppeteer`: control a browser, click elements, take screenshots, and scrape web pages. All are available as npm packages requiring no server code.

10. MCP Cheat Sheet

Building a Server — Minimal Template

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
import asyncio

app = Server("my-server")

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="my_tool",
            description="What this tool does – be specific",

            inputSchema={"type": "object", "properties": {"param": {"type": "string"}}, "required": ["param"]}
        )
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "my_tool":
        result = do_something(arguments["param"])
        return [TextContent(type="text", text=str(result))]

async def main():
    async with stdio_server() as (r, w):
        await app.run(r, w, app.create_initialization_options())

if __name__ == "__main__": asyncio.run(main())
```

Claude Desktop Config — Template

```
{
  "mcpServers": {
    "my-server": {
      "command": "python", "args": ["path/to/server.py"],
      "filesystem": {
        "command": "npx", "args": ["-y",
"@modelcontextprotocol/server-filesystem", "/path"]},
      "brave-search": {
        "command": "npx", "args": ["-y",
"@modelcontextprotocol/server-brave-search"], "env": {"BRAVE_API_KEY": "key"}},
      "github": {
        "command": "npx", "args": ["-y",
"@modelcontextprotocol/server-github"], "env": {"GITHUB_PERSONAL_ACCESS_TOKEN":
"token"}}
    }
  }
}
```

Using MCP Tools in Python — Template

```
from langchain_mcp_adapters.tools import load_mcp_tools
from langchain_groq import ChatGroq
from langgraph.prebuilt import create_react_agent
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
import asyncio

async def main():
    params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(params) as (r, w):
        async with ClientSession(r, w) as session:
            await session.initialize()
            tools = await load_mcp_tools(session)
            agent = create_react_agent(ChatGroq(model="llama3-8b-8192"), tools)
            res = await agent.ainvoke({"messages":
[{"role": "user", "content": "your question"}]})
            print(res["messages"][-1].content)

asyncio.run(main())
```

Key Concepts at a Glance

Concept	One-Line Summary
MCP	Universal standard for connecting LLMs to tools — like USB for AI
MCP Client	The LLM app that consumes tools (Claude Desktop, Python app)
MCP Server	The program that exposes tools via the MCP protocol
Tools	Callable functions with parameters — the LLM invokes these
Resources	Data the LLM can read by URI (files, database tables)
Prompts	Reusable prompt templates the LLM can invoke
inputSchema	JSON Schema defining parameters — tells LLM how to call the tool
list_tools()	Returns catalog of available tools to the MCP client
call_tool()	Executes a tool when the LLM requests it
stdio transport	Local — client spawns server as subprocess via stdin/stdout
HTTP/SSE transport	Remote — server runs as HTTP service, accessible over network
ReAct agent	Reason + Act loop — uses tools iteratively to answer questions

End of MCP Complete Guide